

Explanation

Pawan Mali

June 2026

1 Provided Repository structure Explanation

I analyzed the local folder. It contains two ROS 2 workspaces:

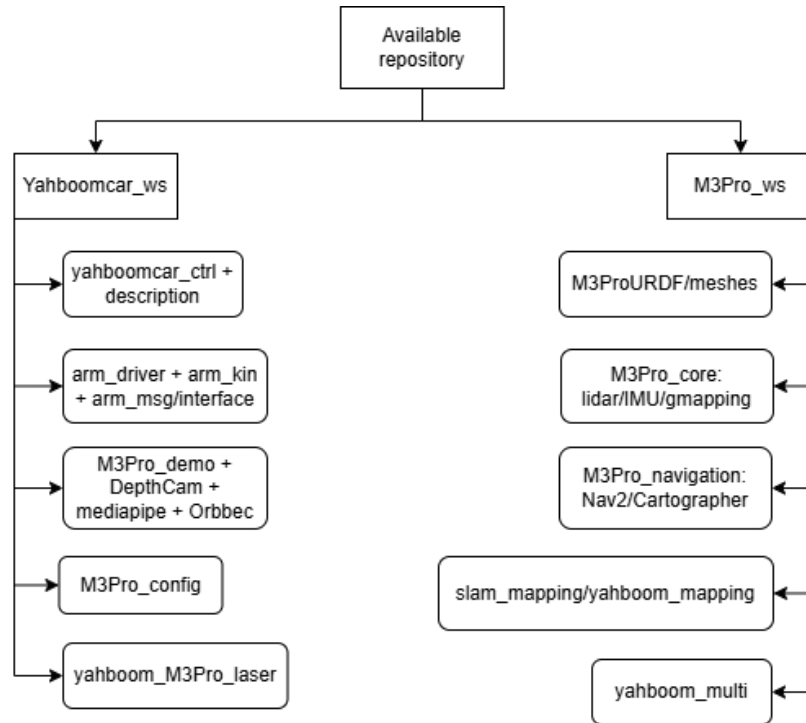


Figure 1: Overview of the downloaded repository structure

1.1 yahboomcar_ws/src

- **yahboomcar_ctrl**: Keyboard and joystick teleoperation. Important files include `yahboom_keyboard.py` and `yahboom_joy_M3Pro.py`.
- **yahboom_M3Pro_description**: Contains the robot URDF, meshes, and RViz model display files.
- **M3Pro_config**: MoveIt2 configuration package containing arm planning settings and fake `ros2_control` configuration.
- **M3Pro_demo**: Includes computer vision and manipulation demonstrations such as AprilTag detection, color tracking, KCF tracking, grasping, and line-following.
- **yahboom_M3Pro_laser**: Lidar-based applications including obstacle avoidance, obstacle warning, and target tracking.
- **yahboom_M3Pro_DepthCam**: Depth camera examples including distance measurement, depth-color visualization, and edge detection.
- **OrbbecSDK_ROS2**: ROS 2 driver package for Orbbec/Dabai depth cameras.
- **arm_interface**, **arm_msgs**: Custom messages and services used for robotic arm communication.
- **arm_kin**: Forward and inverse kinematics services implemented using KDL.
- **arm_driver**: Serial communication driver for the robotic arm using the Rosmaster library.

1.2 M3Pro_ws/src

- **M3Pro**: Robot URDF, meshes, and model display launch files.
- **M3Pro_core**: Core robot packages including laser merging, laser filtering, IMU filtering, and GMapping support.
- **M3Pro_navigation**: Navigation stack configuration including Nav2, Cartographer localization, and RViz navigation setups.
- **slam_mapping**, **yahboom_mapping**: Packages used for SLAM and map generation/saving.
- **ekf_bringup**: Robot localization package that fuses `/odom_raw` and `/imu/data` using an Extended Kalman Filter (EKF).
- **calibration**: Linear and angular calibration utilities based on `/cmd_vel` and TF data.

- **yahboom_multi**: Multi-robot navigation and coordination configurations.
- **patrol**: Laser-based autonomous patrol behavior implementation.
- **largemodel**, **largemodel_arm**: AI, voice interaction, and computer vision application examples.

2 Complete Hardware and software based pipeline

2.1 Hardware pipeline

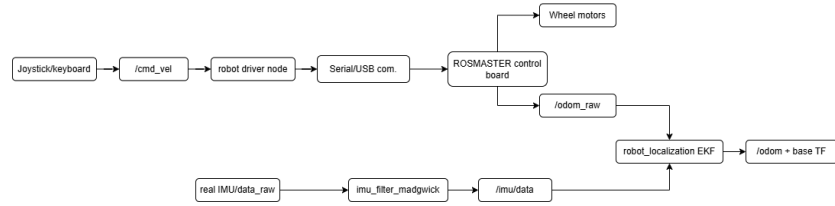


Figure 2: Hardware control pipeline overview

In the hardware-based system, movement commands are generated by a keyboard, joystick, and published on the `/cmd_vel` topic. These commands are sent through the robot driver and serial communication interface to the ROSMASTER control board, which controls the motor driver and wheel motors to move the robot. Wheel encoders provide raw odometry data, while the IMU publishes motion data. The IMU data is processed using the Madgwick filter to obtain a cleaner orientation estimate. Finally, the robotlocalization EKF uses the odometry and IMU data to produce accurate robot pose information (`/odom`) transformation, which are used by RViz2, SLAM, and Nav2 for localization and navigation.

2.2 Software pipeline

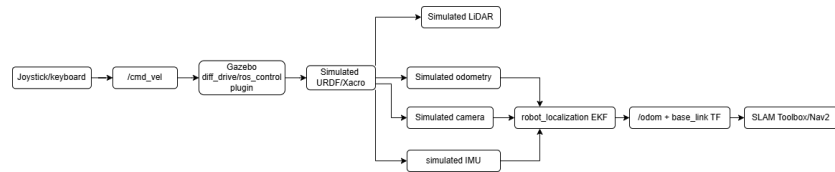


Figure 3: Software control pipeline overview

In the software based system, movement commands are generated by a keyboard and are published on the `/cmd_vel` topic. Instead of being sent to a

physical robot, these commands are processed by a Gazebo `diff_drive` or `ros2_control` plugin, which moves the simulated robot model defined by the URDF/Xacro files. As the robot moves within the virtual environment, Gazebo generates simulated sensor data, including odometry, IMU, LiDAR, and camera information. The odometry and IMU data are used by the `robot_localization` Extended Kalman Filter (EKF) to produce accurate robot pose estimates and the `odom` to `base_link` transformation. The simulated LiDAR data is used by SLAM Toolbox and Nav2 for mapping, localization, and path planning, while the simulated camera data can be utilized for computer vision tasks such as object detection and obstacle avoidance. This simulation-based architecture enables the development, testing, and validation of robotic algorithms without requiring physical hardware.

3 Task 1: Simulation based teleoperation

For this first step, the pipeline contains only:

- M3Pro base model
- Gazebo physics
- robot state publisher
- Keyboard or joystick teleoperation
- `cmd vel`
- Gazebo movement plugin

3.1 keyboard teleoperation

Previous hardware pipeline:

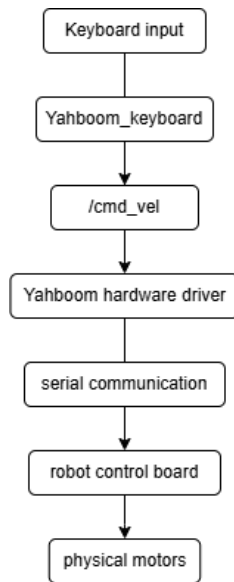


Figure 4: Hardware pipeline overview

The existing `yahboomcar_keyboard.py` node can be used without any changes.

New software pipeline:

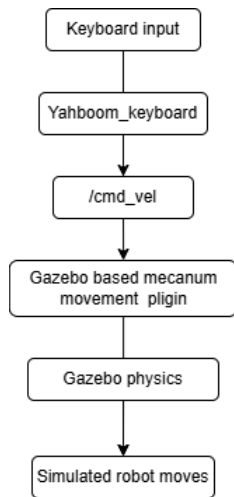


Figure 5: Software pipeline overview

4 understand the repo structure and debug the errors

What's actually in this repo

- Two workspaces: yahboomcar_ws (arm driver, robot description/URDF, MoveIt2 config, laser, depth camera) and M3Pro_ws (navigation2, SLAM, calibration, and a big “largemodel” AI/voice stack).
- The Docker_M3Pro-*.sh scripts pull an image from 192.168.2.51:5000 — that's a private LAN registry inside Yahboom's factory network, not reachable from your our system at all.
- Hardware drivers are hardcoded, e.g. arm_driver.py does Rosmaster("/dev/ttyUSB0") using a proprietary Rosmaster_Lib pip package. Without hardware, this will throw `SerialException: could not open port /dev/ttyUSB0`.
- The only working "no-hardware" launch files are plain URDF viewers (display.launch.py), which just start robot_state_publisher + joint_state_publisher.

4.1 RViz2 / Robot Description Debugging

During comiling the repo, I encountered some errors related to missing packages and dependencies. To resolve these issues, I followed these steps:

```
[robot_state_publisher-2] [INFO] [1783430310.599880042] [robot_state_publisher]:  
got segment rwheel1  
[robot_state_publisher-2] [INFO] [1783430310.599882271] [robot_state_publisher]:  
got segment rwheel2  
[joint_state_publisher-1] [INFO] [1783430310.789573185] [joint_state_publisher]:  
Waiting for robot_description to be published on the robot_description topic...  
^C[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)  
[robot_state_publisher-2] [INFO] [1783430729.461275397] [rclcpp]: signal_handler  
(SIGINT/SIGTERM)  
[INFO] [robot_state_publisher-2]: process has finished cleanly [pid 29831]  
[INFO] [joint_state_publisher-1]: process has finished cleanly [pid 29829]  
pawan@pawan-LQ-15IRX10: ~/yahboomcar_ws_test/yahboomcar_ws$ grep -rn "dofbot_M3P  
ro_description" src/  
src/yahboom_M3Pro_description/urdf/M3Pro.urdf:1098:         filename="package:/  
/dofbot_M3Pro_description/meshes/arm_base_Link.STL" />  
src/yahboom_M3Pro_description/urdf/M3Pro.urdf:1113:         filename="package:/  
/dofbot_M3Pro_description/meshes/arm_base_Link.STL" />  
src/yahboom_M3Pro_description/urdf/.ipynb_checkpoints/M3Pro-checkpoint.urdf:1098  
:         filename="package://dofbot_M3Pro_description/meshes/arm_base_Link.STL"  
/>  
src/yahboom_M3Pro_description/urdf/.ipynb_checkpoints/M3Pro-checkpoint.urdf:1113  
:         filename="package://dofbot_M3Pro_description/meshes/arm_base_Link.STL"  
/>  
pawan@pawan-LQ-15IRX10: ~/yahboomcar_ws_test/yahboomcar_ws$
```

Figure 6: URDF errors

The robot model initially showed an error because the package name in the URDF file was incorrect. The error was found at lines 1098 and 1113. According to the correction, rebuilt the workspace using `colcon build`, and launched the model again. After the correction, the robot appeared successfully without errors.

4.2 RViz2 / Controlling the Robot movement

I tested the Chassis Control Course, especially section 3.3.1 - Controlling the Car's Speed, without connecting the real robot hardware.

After launching the robot description using `display.launch.py`. The URDF model loaded correctly in RViz, and the robot was visible without errors. I then checked the active ROS 2 nodes and topics. Only visualization-related nodes such as robot state publisher, joint state publisher, and RViz were running. There was no chassis driver or motor control node.

```
ns. It should start with a lower case letter and only contain lower case letters
, digits, underscores, and dashes.
Starting >>> yahboom_M3Pro_description
Finished <<< yahboom_M3Pro_description [0.61s]

Summary: 1 package finished [0.79s]
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ source install/setup
.bash
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ ros2 node list
/joint_state_publisher
/robot_state_publisher
/rviz
/transform_listener_impl_62f43ba51730
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ ros2 topic list
/clicked_point
/goal_pose
/initialpose
/joint_states
/parameter_events
/robot_description
/rosout
/tf
/tf_static
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$
```

Figure 7: Node and Topic List

Next, a velocity command was published to the `/cmd_vel` topic:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.1, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" --once
```

The command continuously showed:

```
Waiting for at least 1 matching subscription(s)...
```

This happened because no node was subscribed to `/cmd_vel`. On the real robot, the chassis driver receives this command, sends motor speed values to the STM32 controller through serial communication, reads encoder data, and publishes odometry. Since the hardware and chassis driver were not available, the velocity command had no receiver.

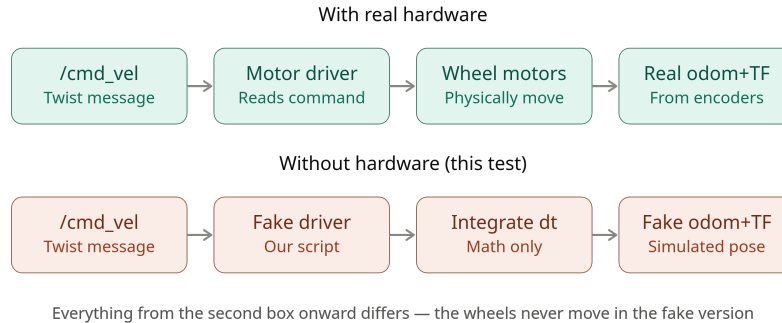


Figure 8: Real vs. Fake Control Pipeline

To test the chassis-control command without the real robot hardware, a temporary node called `fake_odom_driver.py` was added. The original command publishes velocity values on `/cmd_vel`, but no real motor driver is available to receive them. Therefore, the robot cannot move in RViz using only `robot_state_publisher`.

The fake driver works as a simple software replacement for the hardware driver. It subscribes to `/cmd_vel` and reads the commanded forward, sideways, and rotational velocities. These values are stored as:

```
linear.x  -> forward/backward movement
linear.y  -> sideways movement for mecanum wheels
angular.z -> rotation
```

The node then calculates the robot's new position over time using simple dead-reckoning. It updates the robot's x , y , and heading angle based on the latest velocity command. After calculating the pose, it publishes an odometry message on `/odom` and broadcasts the TF transformation:

```
odom -> base_link
```

RViz uses this TF transformation to update the position of the complete robot model. The robot links and joints are still displayed by `robot_state_publisher`, while the fake driver changes the position of the robot base.